

PEC2 - Segunda prueba de evaluación continuada

Presentación

Después de conocer los sistemas de representación de la información, esta PEC se focaliza en los circuitos lógicos combinacionales. Las funciones lógicas nos permiten describir la funcionalidad de un circuito, y mediante las puertas lógicas y los bloques combinacionales podemos implementarlas en un circuito. Todo esto no se podría hacer sin conocer mecanismos de minimización, como mapas de Karnaugh, que nos permiten reducir las dimensiones del circuito que se tiene que implementar.

Competencias

- Conocer y saber aplicar el álgebra de Boole para la manipulación de funciones lógicas.
- Tener nociones tecnológicas de los circuitos digitales y entender la relación entre los circuitos digitales y las funciones lógicas.
- Conocer y saber utilizar las puertas lógicas y los módulos combinacionales en el diseño de circuitos lógicos.

Objetivos

- Saber aplicar las operaciones básicas y los axiomas del álgebra de Boole.
- Saber representar las funciones lógicas mediante tablas de verdad.
- Saber representar las funciones lógicas mediante expresiones algebraicas.
- Saber analizar un circuito combinacional.
- Saber realizar un cronograma a partir de un circuito digital combinacional.
- Saber sintetizar una función a dos niveles.
- Saber diseñar un circuito combinacional sencillo a partir de los bloques combinacionales de los materiales.

Recursos

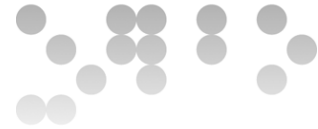
Los recursos que se recomienda usar para esta PEC son los siguientes:

Básicos: El módulo 3 de los materiales.

Complementarios: KeMap, VerilCIRC, VerilChart y el Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.
- La valoración está indicada en cada uno de los sub-apartados.



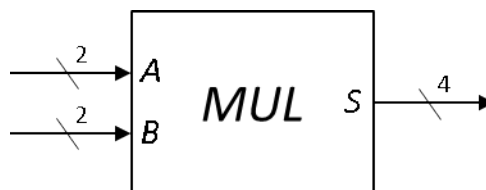
Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF usando una de las plantillas proporcionadas conjuntamente con este enunciado.
- Se tiene que entregar a través de la aplicación de Entrega **y registro de EC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **28 de octubre** (a las 24 horas).

Enunciado de la PEC

Ejercicio 1 [15 %]

Se quiere diseñar un circuito multiplicador *MUL* como el que muestra la figura siguiente:



Donde la entrada *A* contiene un número binario natural de dos bits, y la entrada *B* contiene un número entero codificado en signo y magnitud y dos bits. La salida *S* tiene que mostrar el resultado de la multiplicación en formato de Ca2 y 4bits.

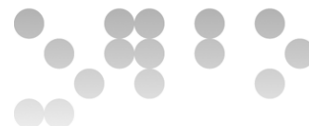
Se pide:

- [5 %] Rellenad las tablas siguientes con la codificación en decimal de los diferentes valores que pueden tomar las señales *A* y *B*, respectivamente, de acuerdo con su codificación.

En el caso de las señales *A*, simplemente aplicamos TFN para hacer el cambio de base de binario a decimal, mientras que en caso de la señal *B* interpretamos el primer bit como signo y el segundo como magnitud.

<i>a1</i>	<i>a0</i>	decimal
0	0	0
0	1	1
1	0	2
1	1	3

<i>b1</i>	<i>b0</i>	decimal
0	0	+0
0	1	+1
1	0	-0
1	1	-1



- b) [5 %] Rellenad la tabla de verdad siguiente que implementa la funcionalidad descrita.

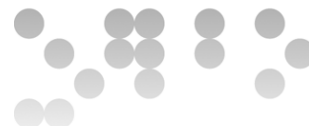
Para rellenar la tabla hacemos la multiplicación del valor de la señal A por el de la señal B en decimal y representamos el resultado en $Ca2$.

$a1$	$a0$	$b1$	$b0$	$s3$	$s2$	$s1$	$s0$
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	0	0	0
1	0	1	1	1	1	1	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	0	0	0
1	1	1	1	1	1	0	1

- c) [5 %] Expresad sin simplificar la función de salida $s3(a1, a0, b1, b0)$ en forma de suma de minterminos.

Los minterminos de una función están conformados por los productos de las variables para los cuales la función vale 1. La salida $s3$ sólo vale 1 para las combinaciones 0111, 1011 y 1111 de las variables $a1$, $a0$, $b1$ y $b0$. Así, su expresión como suma de minterminos es:

$$s3(a1, a0, b1, b0) = a1' \cdot a0 \cdot b1 \cdot b0 + a1 \cdot a0' \cdot b1 \cdot b0 + a1 \cdot a0 \cdot b1 \cdot b0$$



Ejercicio 2 [20 %]

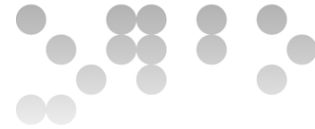
- a) [10 %] Dada la tabla de verdad siguiente, sintetizad de manera mínima a dos niveles la función f mediante el método de Karnaugh, e implementad el resultado con puertas lógicas:

a	b	c	d	f
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

NOTA: Tenéis disponible el ejercicio en KeMAP para verificarlo. En cuanto al mapa de Karnaugh, para evitar un mal uso de KeMAP, en caso de que se hagan más de 5 intentos la nota del ejercicio se reducirá al 50 %. Os recomendamos hacer pruebas con KeMAP con otros ejercicios antes de hacer los de la PEC.

El mapa de Karnaugh de la función f , con un agrupamiento posible, es el siguiente:

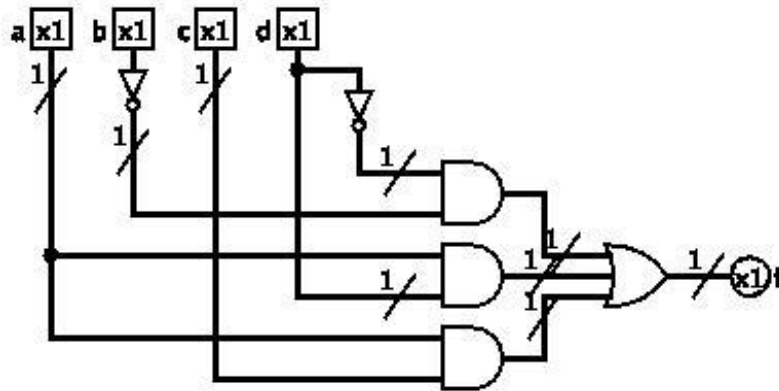
ab \ cd	00	01	11	10
00	1	0	0	1
10	0	0	1	1
11	0	0	1	1
10	1	0	1	1



del cual obtenemos la siguiente expresión mínima:

$$f = b' \cdot d' + a \cdot d + a \cdot c$$

El circuito que implementa esta función con puertas lógicas es el siguiente:



- b) [10 %] Dada la tabla de verdad siguiente, sintetizad de manera mínima a dos niveles la función g mediante el método de Karnaugh, e implementad el resultado con puertas lógicas:

a	b	c	d	g
0	0	0	0	X
0	0	0	1	X
0	0	1	0	X
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	X
0	1	1	1	X
1	0	0	0	X
1	0	0	1	0
1	0	1	0	1
1	0	1	1	X
1	1	0	0	0
1	1	0	1	X
1	1	1	0	0
1	1	1	1	0

El mapa de Karnaugh de la función g , con un agrupamiento posible, es el siguiente:

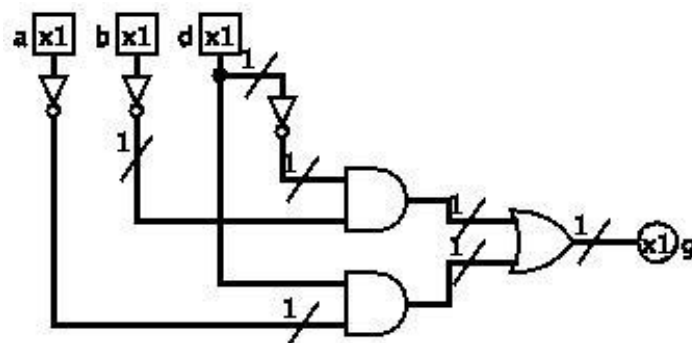


ab \ cd	00	01	11	10
00	X	0	0	X
10	X	1	X	0
11	1	X	0	X
10	X	X	0	1

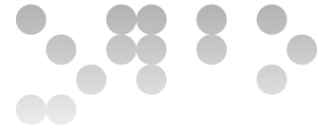
del cual obtenemos la siguiente expresión mínima:

$$g = b' \cdot d' + a' \cdot d$$

El circuito que implementa esta función con puertas lógicas es el siguiente:



En este caso podemos hacer un agrupamiento alternativo que también optimiza el circuito y que es el siguiente:

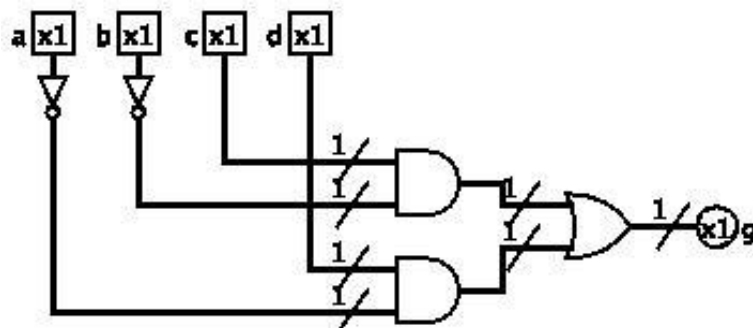


ab \ cd	00	01	11	10
00	X	0	0	X
10	X	1	X	0
11	1	X	0	X
10	X	X	0	1

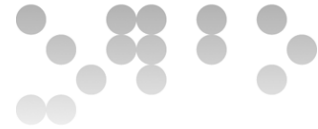
del cual obtenemos la siguiente expresión mínima:

$$g = c \cdot b' + a' \cdot d$$

El circuito que implementa esta función con puertas lógicas es el siguiente:



NOTA: Tenéis disponible el ejercicio en KeMAP para verificarlo. En cuanto al mapa de Karnaugh, para evitar un mal uso de KeMAP, en caso de que se hagan más de 5 intentos la nota del ejercicio se reducirá al 50 %. Os recomendamos hacer pruebas con KeMAP con otros ejercicios antes de hacer los de la PEC.

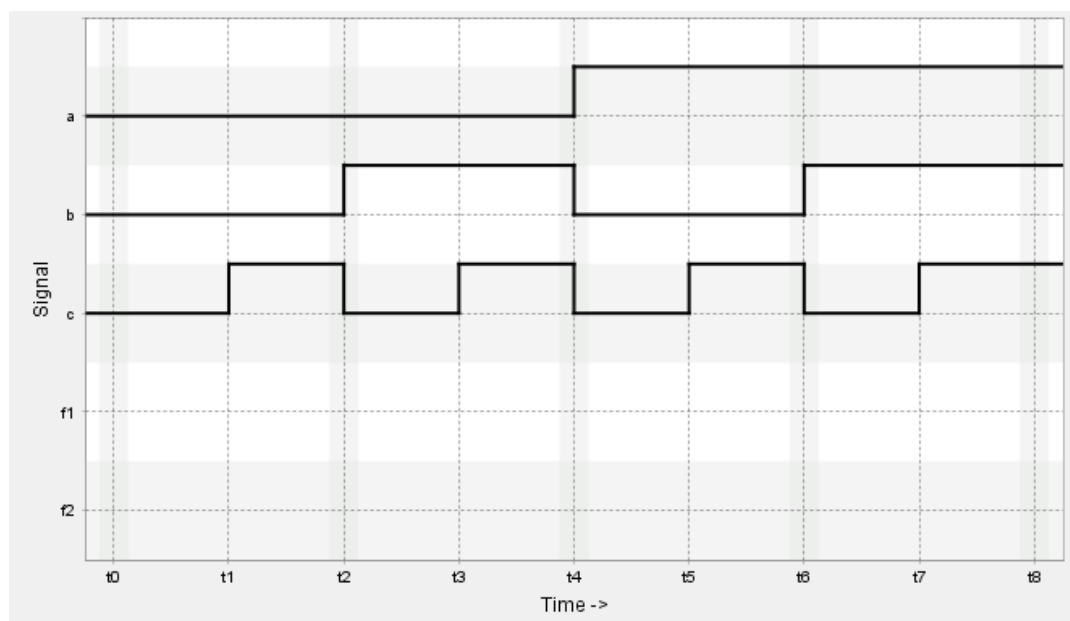


Ejercicio 3 [10 %]

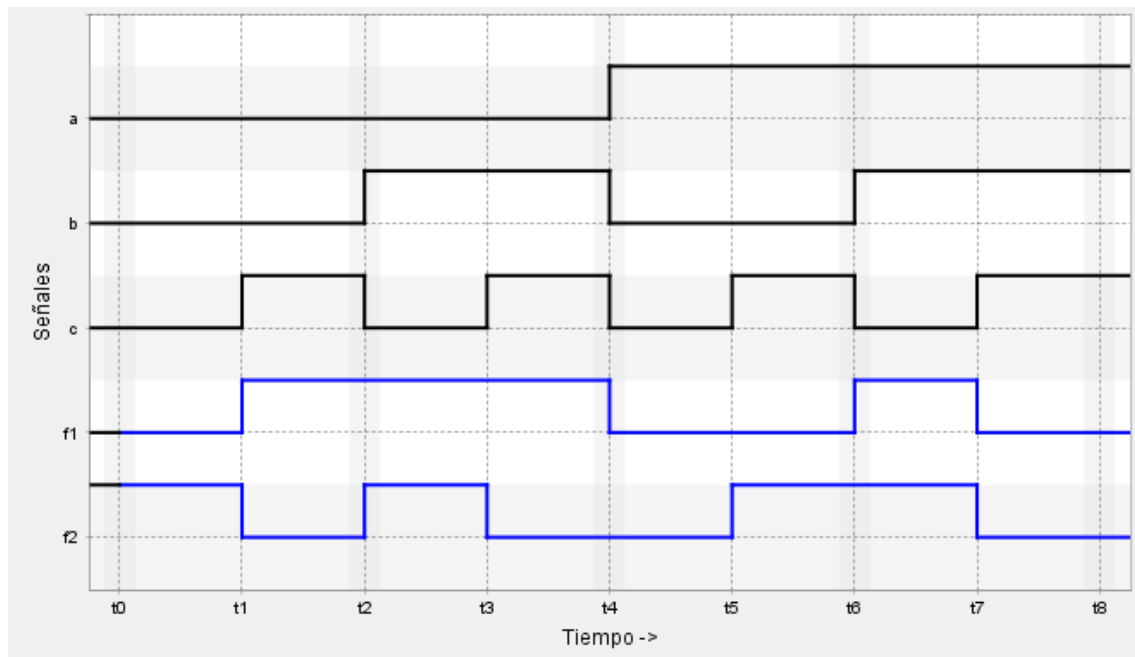
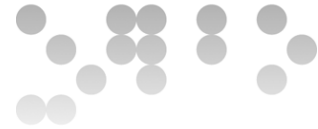
Dada la tabla de verdad siguiente:

<i>a</i>	<i>b</i>	<i>c</i>	<i>f1</i>	<i>f2</i>
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	1	0
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	0	0

Completa el cronograma siguiente:



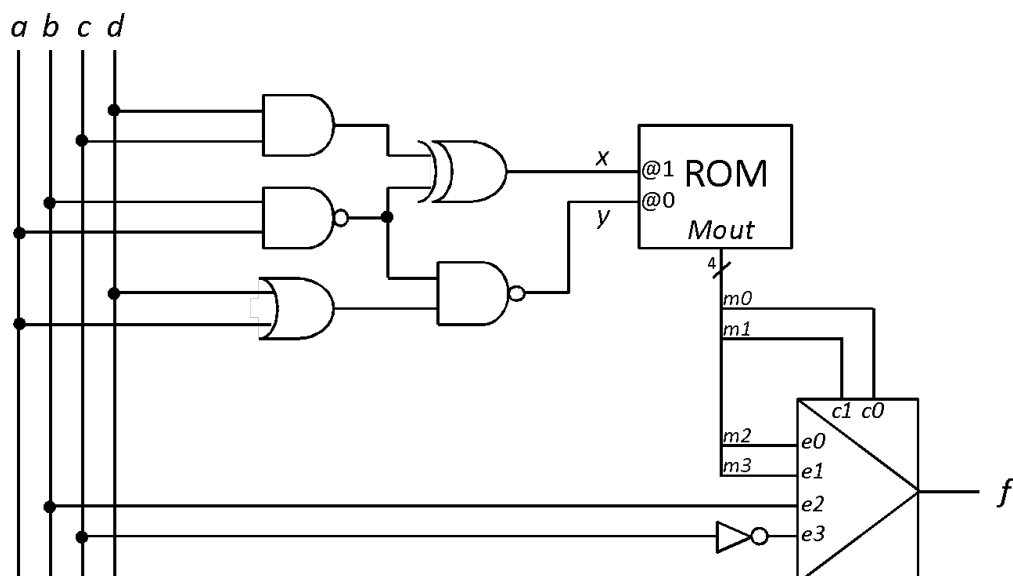
Las salidas f_1 y f_2 del cronograma se determinan haciendo uso de la tabla de verdad a partir de la combinación concreta de las señales de entrada que hay en cada momento.

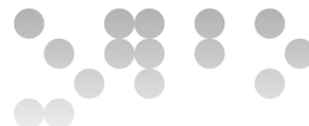


NOTA: Tenéis disponible el ejercicio en VeriLCHART.

Ejercicio 4 [25 %]

Dado el circuito lógico combinacional siguiente:





Donde el contenido de la memoria ROM es:

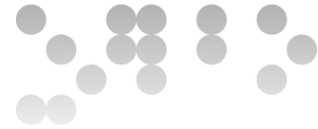
@	M[@]
0	0001
1	1011
2	1101
3	1100

Se pide que rellenéis la tabla de verdad para la función $f(a,b,c,d)$, calculando previamente los valores intermedios indicados en la tabla.

Los valores de la tabla de verdad se han obtenido de la siguiente forma:

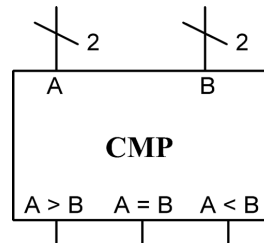
1. Se ha calculado el valor de salida del primer nivel de puertas lógicas (incluyendo $a+d$)
2. Se ha calculado el valor el valor de x e y como salidas del segundo nivel de puertas lógicas
3. Se usan las señales x e y como direcciones de la memoria ROM para determinar los valores de las señales $m3,...,m0$
4. Se usan las señales $m1$ y $m0$ como señales de selección del multiplexor y calcular el valor de f .

a	b	c	d	c·d	(a·b)'	x	y	m3	m2	m1	m0	f
0	0	0	0	0	1	1	1	1	1	0	0	1
0	0	0	1	0	1	1	0	1	1	0	1	1
0	0	1	0	0	1	1	1	1	1	0	0	1
0	0	1	1	1	1	0	0	0	0	0	1	0
0	1	0	0	0	1	1	1	1	1	0	0	1
0	1	0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	0	1	1	1	1	1	0	0	1
0	1	1	1	1	1	0	0	0	0	0	1	0
1	0	0	0	0	1	1	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1	1	0	1	1
1	0	1	0	0	1	1	0	1	1	0	1	1
1	0	1	1	1	1	0	0	0	0	0	1	0
1	1	0	0	0	0	0	1	1	0	1	1	1
1	1	0	1	0	0	0	1	1	0	1	1	1
1	1	1	0	0	0	0	1	1	0	1	1	0
1	1	1	1	1	0	1	1	1	1	0	0	1



Ejercicio 5 [30 %]

Dado un bloque comparador *CMP* de números **naturales** de dos bits, como el que muestra la figura siguiente:



Se pide:

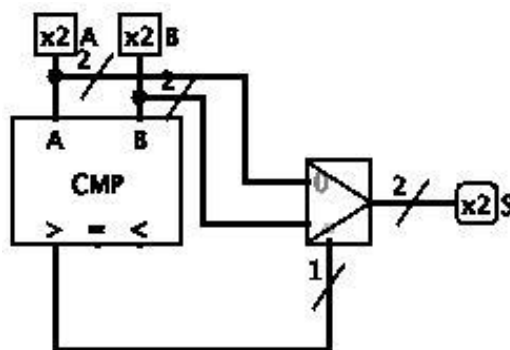
- a) [10 %] Diseñad un nuevo bloque, denominado *COM2*:



que usando un comparador *CMP* y otros bloques o puertas combinatoriales, saque por la salida *S* el valor del número más pequeño (*A* o *B*). Por ejemplo:

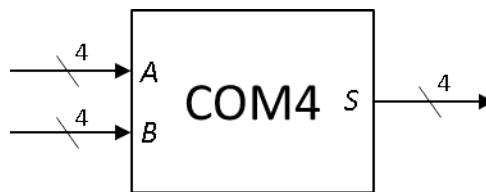
- Si $A = 3$ y $B = 1$, entonces $S = 1$;
- Si $A = 2$ y $B = 2$, entonces $S = 2$.

En este caso usamos un comparador (*CMP*) para generar la señal de selección de un multiplexor de dos entradas de dos bits. En la entrada 0 del multiplexor conectamos la señal *A* y en la entrada 1 la señal *B*, así si *A* es mayor que *B* se selecciona la entrada 1 del multiplexor (*B*) y sino la entrada 0 (*A*).





b) [20 %] Diseñad ahora un nuevo bloque, denominado *COM4*:

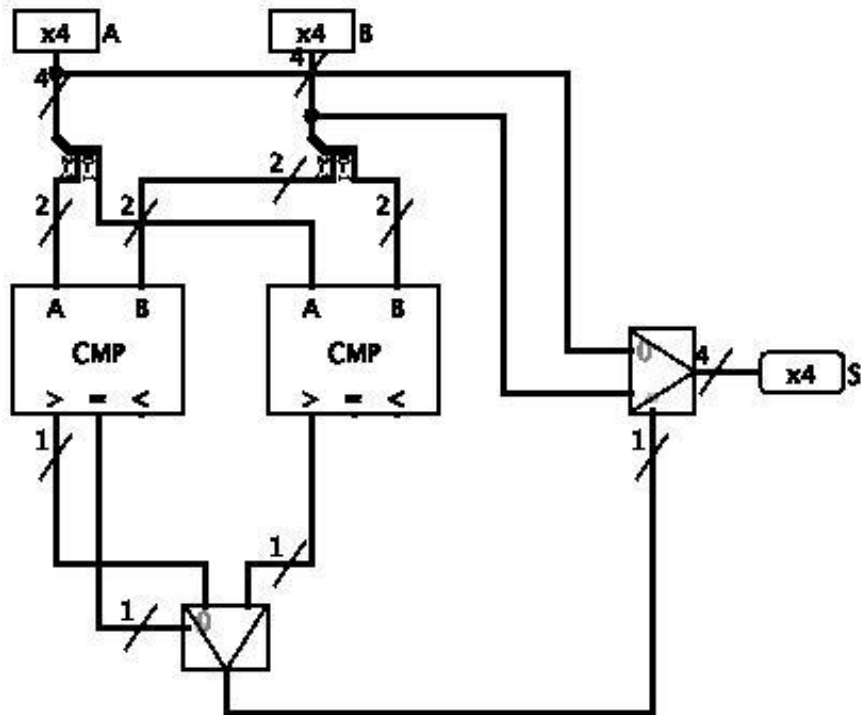


que usando uno o varios comparadores *CMP* (**de dos bits**) y otros bloques o puertas combinacionales, saque por la salida *S* el valor del número más pequeño (*A* o *B*), de cuatro bits cada uno.

En este caso separamos los cuatro bits de las entradas *A* y *B*, y comparamos los dos más significativos de cada número y los dos menos significativos. Si los bits más significativos son diferentes, entonces serán ellos los que determinan qué número es más pequeño. Ahora bien, si los bits más significativos son iguales, son los menos significativos los que determinan cuál es el número más pequeño.

Así, usamos dos comparadores (*CMP*) para comparar los bits más y menos significativos de *A* y *B*, respectivamente. Las salidas $A > B$ de estos comparadores se conectan a un multiplexor de dos entradas de un bit. Por este multiplexor usamos como señal de selección la salida $A = B$ del comparador que tiene como entrada los bits más significativos de *A* y *B*. En consecuencia, si los bits más significativos de *A* y *B* son diferentes, dejamos pasar la señal del comparador que tiene como entrada los bits más significativos y, si son iguales, dejamos pasar la señal del comparador que tiene como entrada los bits menos significativos.

La salida de este multiplexor sirve como señal de selección de otro multiplexor de dos entradas de 4 bits. En la entrada 0 del multiplexor conectamos la señal *A* y en la entrada 1 la señal *B*, así, si *A* es mayor que *B* se selecciona la entrada 1 del multiplexor (*B*) y sino la entrada 0 (*A*).



NOTA: Tenéis disponible el ejercicio en VerilCIRC, sin límite en el número de intentos para comprobar vuestra solución.